MDM Outputs

Experiment 2

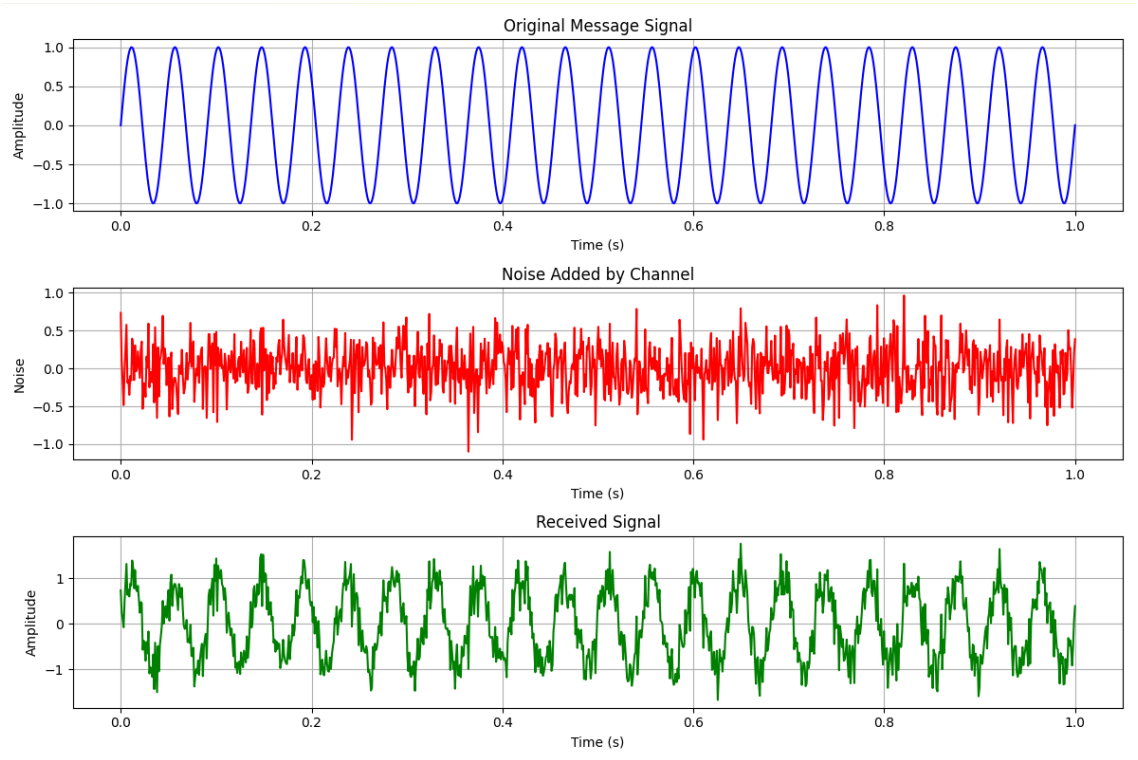
Code:

```
# Simulation of Basic Communication System (Tx-Channel-Rx)
import numpy as np
import matplotlib.pyplot as plt
# ------------------------
# Transmitter
# ------------------------
# Time axis
t = np.linspace(0, 1, 1000)
# Original message signal
frequency = 22 # Hz
message = np.sin(2 * np.pi * frequency * t)
# ------------------------
# Communication Channel
# ------------------------
# Generate Gaussian Noise
noise = np.random.normal(0, 0.3, len(t))
# Signal after passing through channel
received_signal = message + noise
# ------------------------
# Receiver
# ------------------------
# In this simple simulation, receiver directly receives noisy signal
recovered_signal = received_signal
# ------------------------
# Plotting
# ------------------------
plt.figure(figsize=(12,8))
plt.subplot(3,1,1)
plt.plot(t, message, color='blue')
```

MDM Outputs

```
plt.title("Original Message Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.subplot(3,1,2)
plt.plot(t, noise, color='red')
plt.title("Noise Added by Channel")
plt.xlabel("Time (s)")
plt.ylabel("Noise")
plt.grid(True)
plt.subplot(3,1,3)
plt.plot(t, recovered_signal, color='green')
plt.title("Received Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output:



MDM Outputs

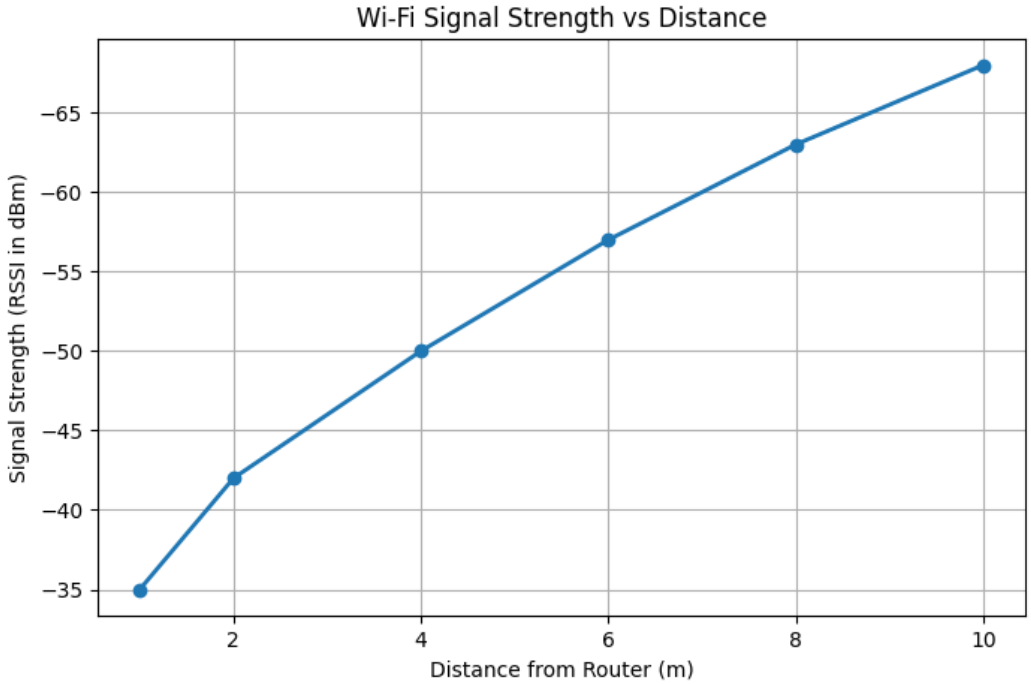
Experiment 4

Code:

```
import matplotlib.pyplot as plt
# Distance from Wi-Fi router (meters)
distance = [1, 2, 4, 6, 8, 10]
# Sample RSSI values (dBm)

rssi = [-35, -42, -50, -57, -63, -68]
plt.figure(figsize=(8,5))
plt.plot(distance, rssi, marker='o', linewidth=2)
plt.title("Wi-Fi Signal Strength vs Distance")
plt.xlabel("Distance from Router (m)")
plt.ylabel("Signal Strength (RSSI in dBm)")
plt.grid(True)
# In RSSI, larger negative values mean weaker signal.
# Invert y-axis so stronger signals appear at the top.
plt.gca().invert_yaxis()
plt.show()
```

Output:



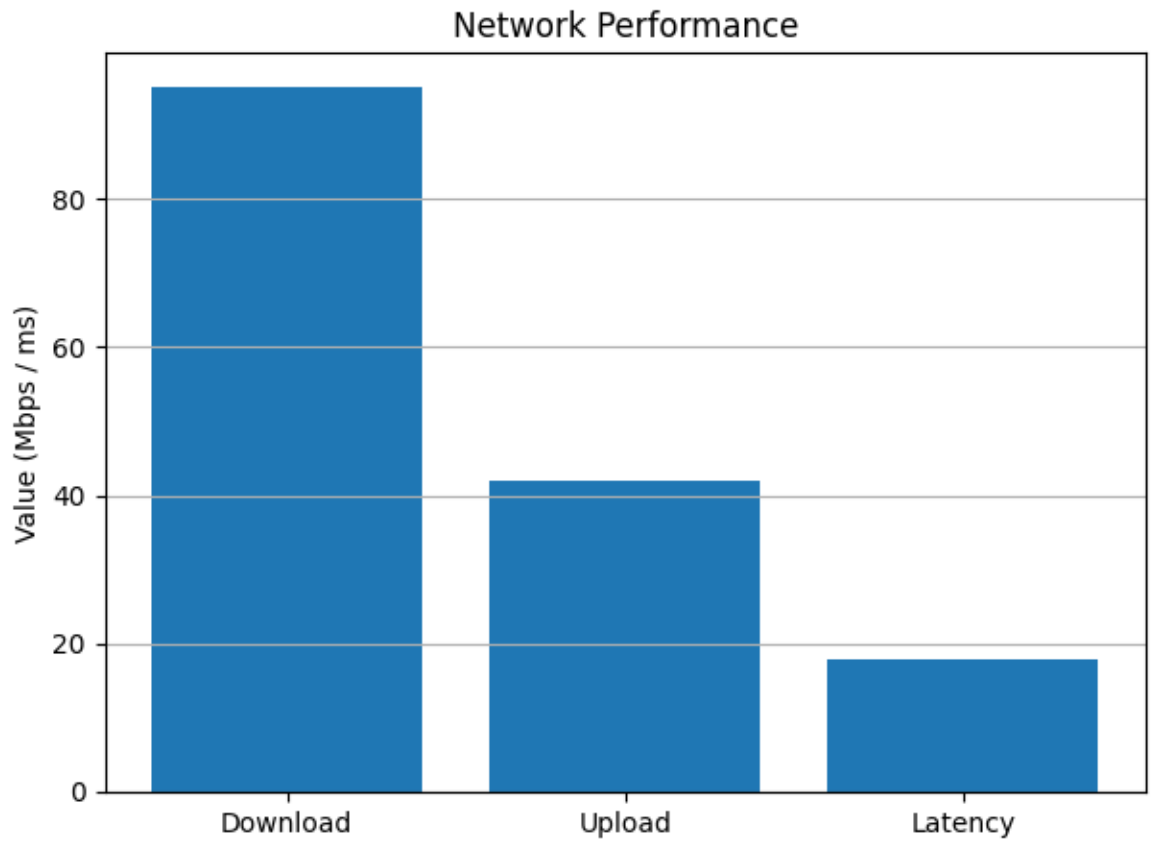
MDM Outputs

Experiment 5

Code:

```
import matplotlib.pyplot as plt
# Sample data
parameters = ['Download', 'Upload', 'Latency']
values = [95, 42, 18]
plt.figure(figsize=(7,5))
plt.bar(parameters, values)
plt.title("Network Performance")
plt.ylabel("Value (Mbps / ms)")
plt.grid(axis='y')
plt.show()
```

Output:



MDM Outputs

Experiment 3

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import RegularPolygon
# ----------------------------------------
# Frequency Reuse Simulation
# ----------------------------------------
fig, ax = plt.subplots(figsize=(10,8))
radius = 1
colors = ['red','green','blue','orange','purple','cyan','yellow']
frequencies = ['A','B','C','D','E','F','G']
positions = [
(0,0),
(2,0),
(4,0),
(1,1.73),
(3,1.73),
(5,1.73),
(2,3.46),
(6,0),
(8,0),
(10,0),
(7,1.73),
(9,1.73),
(11,1.73),
(8,3.46)
]
for i,(x,y) in enumerate(positions):
    freq = i % 7
    hexagon = RegularPolygon(
```

```
        (x,y),
        numVertices=6,
        radius=radius,
        orientation=np.radians(30),
        facecolor=colors[freq],
        edgecolor='black'

    )

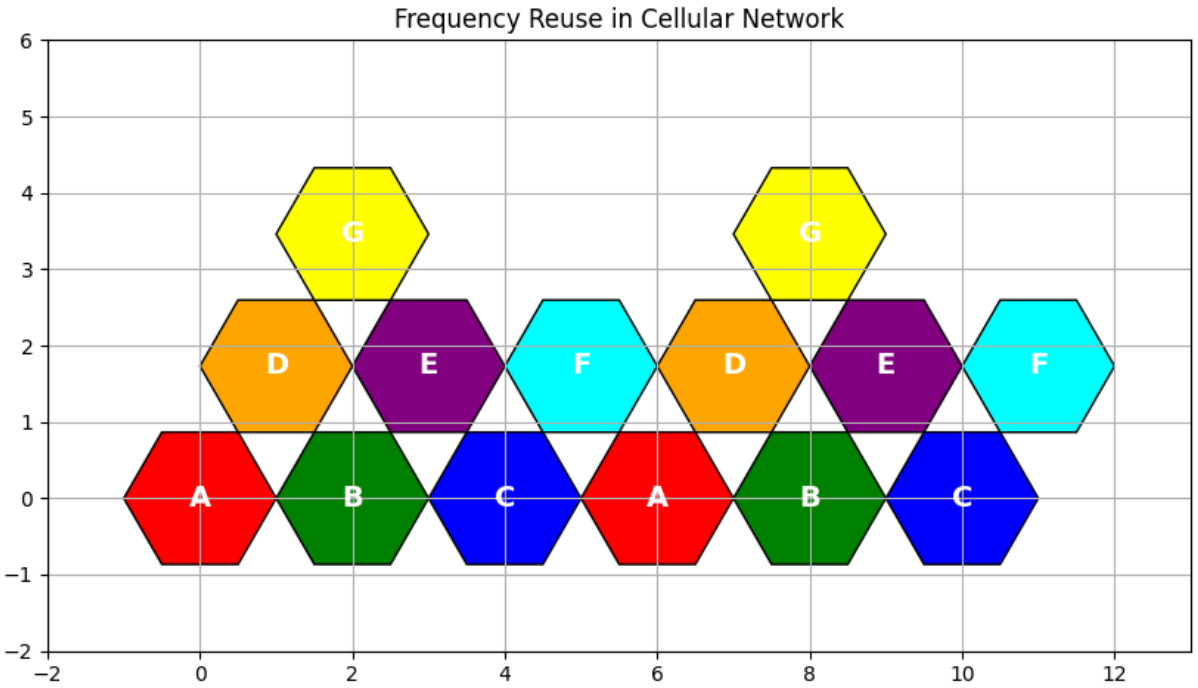
    ax.add_patch(hexagon)

    ax.text(

        x,
        y,
        frequencies[freq],
        fontsize=14,
        ha='center',
        va='center',
        color='white',
        weight='bold'

    )
ax.set_xlim(-2,13)
ax.set_ylim(-2,6)
ax.set_aspect('equal')
ax.set_title("Frequency Reuse in Cellular Network")
plt.grid(True)
plt.show()
```

Output:



MDM Outputs

Experiment 1

Code:

```
import numpy as np
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(14, 7))
regions = [
    ("Gamma\nray", 0.0, 2.0, "#b8e6bd"),
    ("X-ray", 2.0, 3.6, "#9ddfe7"),
    ("UV\nray", 3.6, 4.5, "#f3b183"),
    ("Visible", 4.5, 5.0, "#eeeeee"),
    ("Infrared", 5.0, 7.5, "#69b5e5"),
    ("Micro\nwave", 7.5, 8.8, "#f3d477"),
    ("Radio\nwave", 8.8, 10.5, "#aeb9ca")
]

# Draw spectrum blocks
for name, start, end, color in regions:
    ax.add_patch(
        plt.Rectangle(
            (start, 0),
            end - start,
            0.8,
            facecolor=color,
            edgecolor="white",
            linewidth=2
        )
    )

    ax.text(
        (start + end) / 2,
        0.4,
        name,
        ha="center",
        va="center",
        fontsize=12,
        fontweight="bold"
    )
# ------------------------------------------------
# WAVE
# ------------------------------------------------
x = np.linspace(0, 10.5, 3000)

# Frequency gradually decreases from left to right
cycles = 22 - 19 * (x / 10.5)
# Create smoothly decreasing frequency
phase = 2 * np.pi * (
```

MDM Outputs

```
    22 * x / 10.5
    - 19 * x**2 / (2 * 10.5**2)
)

y = 2.0 + 0.35 * np.sin(phase)

ax.plot(
    x,
    y,
    linewidth=2.5,
    color="#159dcc"
)
ax.text(
    0.9, 2.75,
    "Higher frequency\nShorter wavelength\nHigher energy",
    ha="center",
    va="center",
    fontsize=13,
    fontweight="bold",
    color="#17365d"
)
ax.text(
    9.6, 2.75,
    "Lower frequency\nLonger wavelength\nLower energy",
    ha="center",
    va="center",
    fontsize=13,
    fontweight="bold",
    color="#17365d"
)

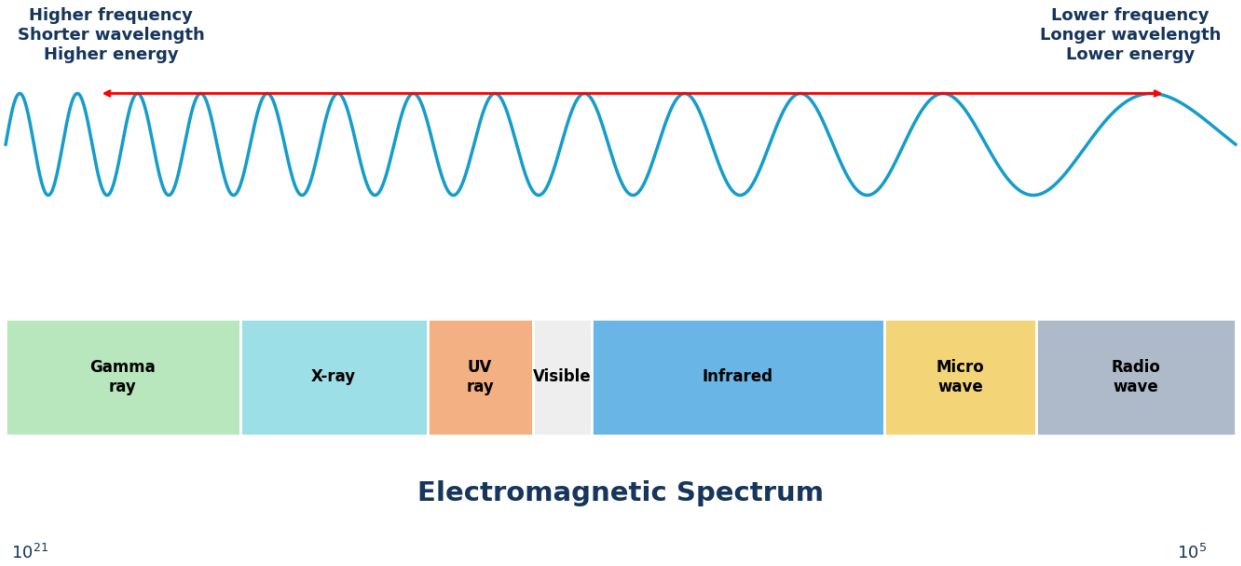
# Red direction arrow
ax.annotate(
    "",
    xy=(9.9, 2.35),
    xytext=(0.8, 2.35),
    arrowprops=dict(
        arrowstyle="<->",
        color="red",
        linewidth=2
    )
)
ax.text(
    5.25,
    -0.45,
    "Electromagnetic Spectrum",
    ha="center",
    fontsize=21,
```

```
    fontweight="bold",
    color="#17365d"
)
# ------------------------------------------------
# FREQUENCY VALUES
# ------------------------------------------------
ax.text(
    0.05, -0.85,
    r"$10^{21}$",
    fontsize=13,
    fontweight="bold",
    color="#17365d"
)
ax.text(
    10.0, -0.85,
    r"$10^{5}$",
    fontsize=13,
    fontweight="bold",
    color="#17365d"
)
# ------------------------------------------------
# FORMATTING
# ------------------------------------------------
ax.set_xlim(-0.2, 10.7)
ax.set_ylim(-1.1, 3.2)

ax.axis("off")

plt.tight_layout()
plt.show()
```

Output:



MDM Outputs

Experiment 8

Code:

```
import matplotlib.pyplot as plt
import numpy as np
# Distance from router in metres
distance = np.array([1, 2, 3, 4, 5, 6, 7, 8])
# Average RSSI obtained from experiment
rssi = np.array([-45, -49, -43, -55, -47, -62, -51, -46])
plt.figure(figsize=(9, 5))
plt.plot(
    distance,
    rssi,
    marker='o',
    linewidth=2
)
plt.xlabel("Distance from Router (m)")
plt.ylabel("Average RSSI (dBm)")
plt.title("Analysis of Multipath Effects")
plt.grid(True)
# More negative RSSI = weaker signal
plt.gca().invert_yaxis()
plt.show()
```

Output:

